USWAG
Unified System for Work and Growth · Normalization Proofs

USWAG Normalization and Dependency Proof (UNF -> 1NF -> 2NF -> 3NF -> BCNF)

Proponents: Bryan Jay M. Lumabas and Claire Pillo. Schema verified in MySQL 8.0 (12 tables).

1. Unnormalized relation (UNF)

`SALE_FORM(sale_id, sale_date, status, seller_id, first_name, last_name, barangay_zone, buyer_account_id, buyer_username, buyer_email, receipt_number, { product_id, product_name, category_id, category_name, unit_of_measure, quantity_sold, unit_price_at_sale })` The braces are a repeating group (one sale has many products).

2. Functional dependencies (non-trivial)

- FD1: sale_id -> sale_date, status, sync_status, seller_id, buyer_account_id, client_id
- FD2: seller_id -> first_name, last_name, barangay_zone, slp_association, account_id
- FD3: buyer_account_id -> buyer_username, buyer_email
- FD4: product_id -> product_name, seller_id, category_id, unit_of_measure, unit_price, current_stock
- FD5: category_id -> category_name
- FD6: (sale_id, product_id) -> quantity_sold, unit_price_at_sale (line_total = quantity_sold * unit_price_at_sale is GENERATED, not stored as a free attribute)
- FD7: sale_id -> receipt_number (1:1, UNIQUE)

Candidate key of the flattened relation: **{sale_id, product_id}** (closure covers every attribute).

3. Decomposition

- **1NF**: remove the repeating group; one row per (sale_id, product_id). R1 = SALE_FORM flattened, key {sale_id, product_id}.
- **2NF**: partial dependencies FD1, FD3, FD7 (on sale_id) and FD4, FD5 (on product_id). Split into `Sales`, `Products`, and `Sale_Details(sale_id, product_id, quantity_sold, unit_price_at_sale)`.
- **3NF**: transitive dependencies sale_id -> seller_id -> barangay_zone, sale_id -> buyer_account_id -> username/email, product_id -> category_id -> category_name. Split into `Sellers`, `Accounts`, `Categories`. Receipts split from Sales (FD7).
- **BCNF**: for every non-trivial FD X -> Y in every final relation, X is a superkey:

Relation	Determinants (all superkeys)
Accounts	account_id, username, email, contact_number, api_token_hash
Sellers	seller_id, account_id, contact_number
Categories	category_id, category_name
Products	product_id, (product_name, seller_id)
Sales	sale_id, (seller_id, client_id)
Sale_Details	sale_detail_id, and (sale_id, product_id) functionally
Receipts	receipt_id, sale_id, receipt_number
Capital_Management	capital_id, seller_id

No determinant is a non-key, so **all relations are in BCNF**. Ledger, Inventory_Transactions, Expenses and Contact_Messages are keyed by surrogate IDs and carry only attributes of that key.

Intentional snapshots (not violations): `Sale_Details.unit_price_at_sale` depends on (sale_id, product_id), not on product_id alone, because prices change over time (FD6). `Capital_Management.current_balance` and `Products.current_stock` are controlled running totals, writable only through procedures/triggers and backed by `Capital_Ledger` / `Inventory_Transactions`.

4. Lossless-join proof (Heath's theorem)

A split of R into R1(X u Y) and R2(X u Z) with X -> Y is lossless because R1 n R2 = X is a key of R1.

- Sales(sale_id, ...) and Sale_Details(sale_id, ...): common attribute sale_id is the key of Sales.
- Products and Sale_Details on product_id: key of Products.
- Sellers and Sales/Products/Expenses/Capital on seller_id: key of Sellers.
- Categories and Products on category_id: key of Categories.
- Accounts and Sales/Receipts/Sellers on account_id: key of Accounts. Each decomposition step joins on the key of one side, so every natural join reconstructs the original tuples with no spurious rows.

5. Dependency-preservation proof

FD1, FD7 -> Sales/Receipts; FD2 -> Sellers; FD3 -> Accounts; FD4 -> Products; FD5 -> Categories; FD6 -> Sale_Details. Each FD has all of its attributes inside one final relation, so F = union of the projections of F on each relation, and no FD needs a join to be enforced.

6. Where physical design goes beyond BCNF

- Foreign keys: ON DELETE RESTRICT for financial history, SET NULL for optional links.
- CHECK constraints: unit_price >= 0, current_stock >= 0, quantity_sold > 0, quantity <> 0, amount >= 0, initial_capital >= 0.

- Triggers guard the running totals (stock, capital) that BCNF alone cannot protect.

© 2026 Bryan Jay Lumabas and Claire Pillo